

Guia de Conceitos de Programação Orientada a Objetos (Java)

Este guia apresenta uma síntese técnica dos fundamentos do paradigma de Programação Orientada a Objetos (POO), transpondo os conceitos teóricos e práticos para a sintaxe Java, sob a perspectiva de desenvolvimento corporativo e robusto.

1. Tabela Comparativa de Paradigmas

A transição da programação estruturada para a POO exige uma mudança na mentalidade de design, focando menos no fluxo sequencial e mais na modelagem de entidades independentes.

Paradigma	Foco e Organização	Dinâmica de Dados
Programação Estruturada	Foca em sequências lógicas, estruturas de controle e processamento de dados de entrada/saída.	Os dados e as funções que os processam são tratados de forma separada e sequencial.
Orientação a Objetos (POO)	Organiza o código em torno de "objetos" (estruturas que unem dados e comportamentos) e suas interações.	Promove modularidade e reutilização, agrupando dados (atributos) e lógica (métodos) em unidades coesas.

2. Tabela Principal: Pilares e Fundamentos de POO

Abaixo, os pilares da POO são detalhados com definições técnicas e implementações idiomáticas em Java.

Conceito	Definição (Baseada na Fonte)	Exemplo de Código Java
Classe	O "molde", "projeto" ou "protótipo" que define a estrutura e os comportamentos de um ente.	<pre>public class Carro { String modelo; void ligar() {} }</pre>
Objeto	Uma "instância" real da classe com identidade própria e valores específicos em memória.	<pre>Carro meuCarro = new Carro();</pre>
Abstração	Simplificação do mundo real para modelos computacionais, focando apenas em aspectos relevantes.	<pre>public class ContaBancaria { private double saldo; private double limite; }</pre>
Encapsulamento	Proteção de dados e exposição mínima de membros através de modificadores de acesso e métodos de acesso.	<pre>private double saldo; public double getSaldo() { return saldo; } public void setSaldo(double s) { this.saldo = s; }</pre>
Herança	Mecanismo que permite a uma subclasse (filho) herdar características e métodos de uma superclasse (pai).	<pre>public class Labrador extends Cao { // herda de Cao }</pre>
Polimorfismo	Capacidade de um objeto se comportar de diferentes formas através da mesma assinatura de método.	<pre>@Override public void emitirSom() { System.out.println("Au Au!"); }</pre>

3. Estrutura Técnica de uma Classe em Java

Abaixo, apresentamos a implementação completa da classe Clientes, integrando atributos privados, construtor explícito e métodos de comportamento conforme as diretrizes técnicas.

```
public class Clientes {
    // Atributos privados (Encapsulamento)
    private String nome;
    private int idade;
    private String endereco;
    private double saldo;

    // Construtor para inicialização de estado
    public Clientes(String nome, int idade, String endereco, double saldo)
    {
        this.nome = nome;
        this.idade = idade;
        this.endereco = endereco;
        this.saldo = saldo;
    }

    // Métodos de Comportamento
    public void comprar(String produto) {
        System.out.println(this.nome + " está comprando o produto: " +
produto);
    }

    public void pagar(double valor) {
        System.out.println(this.nome + " está realizando o pagamento de R$
" + String.format("%.2f", valor));
    }

    public void mostrarInformacoes() {
        System.out.println("Nome: " + this.nome);
        System.out.println("Idade: " + this.idade);
        System.out.println("Endereço: " + this.endereco);
        System.out.println("Saldo em Conta: R$ " + String.format("%.2f",
this.saldo));
    }

    // Getters e Setters (Exemplo de acesso controlado)
    public String getNome() { return nome; }
    public void setNome(String nome) { this.nome = nome; }
    public double getSaldo() { return saldo; }
}
```

4. Instanciação e Identidade de Objetos

A criação de objetos em Java segue um processo rigoroso de alocação de memória e inicialização de estado:

1. **Uso da Palavra-chave new:** Para criar uma instância, utiliza-se obrigatoriamente o operador new, que reserva espaço em memória e invoca o construtor da classe.
2. **Passagem de Argumentos:** Os valores passados ao construtor devem obedecer rigorosamente à ordem e ao tipo dos parâmetros definidos na assinatura do construtor da classe.
3. **Identidade e Estado:** Cada objeto possui sua própria identidade, mesmo que originados do mesmo "molde" (Classe).

Exemplo Prático de Instanciação:

```
// Instância 1: Teresa
Clientes cliente1 = new Clientes("Teresa", 54, "Rua Pindamonhangaba, 30",
50.0);

// Instância 2: João
Clientes cliente2 = new Clientes("João", 19, "Avenida Hortifruti", 700.0);

// Instância 3: Marcos
Clientes cliente3 = new Clientes("Marcos", 25, "Avenida Buriti, 554",
542.0);

// Execução de comportamentos
cliente1.mostrarInformacoes();
cliente2.comprar("Televisão");
cliente2.pagar(1500.00);
```

5. Tabela de Visibilidade (Modificadores de Acesso)

Os modificadores de acesso controlam o nível de exposição dos membros de uma classe, garantindo a integridade dos dados e o cumprimento do contrato de interface.

Modificador	Descrição	Alcance
public	Acesso total e irrestrito.	Visível para qualquer classe em qualquer pacote do projeto.
private	Acesso restrito à implementação.	Visível apenas dentro da própria classe onde foi declarado.
protected	Acesso para herança e pacotes.	Visível para a classe, suas subclasses e classes dentro do mesmo pacote.